In [87]:
```
import pandas as pd
import numpy as np
```

In [88]:
```
marks={
'Name':['Raman','Raman','Raman','Raman','Charan','Charan','Charan'
,'Charan','Ajay','Ajay','Ajay','Ajay','Madhu','Madhu',
'Madhu','Madhu'],
'AT':[1,2,3,4,1,2,3,4,1,2,3,4,1,2,3,4],
'Maths':[22,21,14,np.NaN,20,23,22,19,23,24,12,15,15,18,17,14],
'Science':[21,20,19,np.NaN,17,15,18,20,19,22,25,20,22,21,18,20],
'S.St':[18,17,15,19,22,21,19,17,20,24,19,20,25,25,20,19],
'Hindi':[20,22,24,18,24,25,23,21, 15,17,21,20,22,24,25,20],
'Eng':[21,24,23,np.NaN,19,15,13,16,22,21,23,17,22,23,20,18]}
```

In [89]:
```
marks
```

Out[89]:
```
{'Name': ['Raman',
  'Raman',
  'Raman',
  'Raman',
  'Charan',
  'Charan',
  'Charan',
  'Charan',
  'Ajay',
  'Ajay',
  'Ajay',
  'Ajay',
  'Madhu',
  'Madhu',
  'Madhu',
  'Madhu'],
 'AT': [1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4],
 'Maths': [22, 21, 14, nan, 20, 23, 22, 19, 23, 24, 12, 15, 15, 18, 17, 14],
 'Science': [21, 20, 19, nan, 17, 15, 18, 20, 19, 22, 25, 20, 22, 21, 18, 20],
 'S.St': [18, 17, 15, 19, 22, 21, 19, 17, 20, 24, 19, 20, 25, 25, 20, 19],
 'Hindi': [20, 22, 24, 18, 24, 25, 23, 21, 15, 17, 21, 20, 22, 24, 25, 20],
 'Eng': [21, 24, 23, nan, 19, 15, 13, 16, 22, 21, 23, 17, 22, 23, 20, 18]}
```

In [90]:
```
df1=pd.DataFrame(marks)
df1
```

Out[90]:

	Name	AT	Maths	Science	S.St	Hindi	Eng
0	Raman	1	22.0	21.0	18	20	21.0
1	Raman	2	21.0	20.0	17	22	24.0
2	Raman	3	14.0	19.0	15	24	23.0
3	Raman	4	NaN	NaN	19	18	NaN
4	Charan	1	20.0	17.0	22	24	19.0
5	Charan	2	23.0	15.0	21	25	15.0
6	Charan	3	22.0	18.0	19	23	13.0
7	Charan	4	19.0	20.0	17	21	16.0
8	Ajay	1	23.0	19.0	20	15	22.0
9	Ajay	2	24.0	22.0	24	17	21.0
10	Ajay	3	12.0	25.0	19	21	23.0
11	Ajay	4	15.0	20.0	20	20	17.0
12	Madhu	1	15.0	22.0	25	22	22.0
13	Madhu	2	18.0	21.0	25	24	23.0
14	Madhu	3	17.0	18.0	20	25	20.0
15	Madhu	4	14.0	20.0	19	20	18.0

In [91]:
```
'''Pandas provide a function isnull() to check whether any
value is missing or not in the DataFrame. This function
checks all attributes and returns True in case that
attribute has missing values, otherwise returns False.'''
df1.isnull()
```

Out[91]:

	Name	AT	Maths	Science	S.St	Hindi	Eng
0	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False
3	False	False	True	True	False	False	True
4	False	False	False	False	False	False	False
5	False	False	False	False	False	False	False
6	False	False	False	False	False	False	False
7	False	False	False	False	False	False	False
8	False	False	False	False	False	False	False
9	False	False	False	False	False	False	False
10	False	False	False	False	False	False	False
11	False	False	False	False	False	False	False
12	False	False	False	False	False	False	False
13	False	False	False	False	False	False	False
14	False	False	False	False	False	False	False
15	False	False	False	False	False	False	False

In [92]:
```
df1.dropna()
```

Out[92]:

	Name	AT	Maths	Science	S.St	Hindi	Eng
0	Raman	1	22.0	21.0	18	20	21.0
1	Raman	2	21.0	20.0	17	22	24.0
2	Raman	3	14.0	19.0	15	24	23.0
4	Charan	1	20.0	17.0	22	24	19.0
5	Charan	2	23.0	15.0	21	25	15.0
6	Charan	3	22.0	18.0	19	23	13.0
7	Charan	4	19.0	20.0	17	21	16.0
8	Ajay	1	23.0	19.0	20	15	22.0
9	Ajay	2	24.0	22.0	24	17	21.0
10	Ajay	3	12.0	25.0	19	21	23.0
11	Ajay	4	15.0	20.0	20	20	17.0
12	Madhu	1	15.0	22.0	25	22	22.0
13	Madhu	2	18.0	21.0	25	24	23.0
14	Madhu	3	17.0	18.0	20	25	20.0
15	Madhu	4	14.0	20.0	19	20	18.0

In [93]:
```
df1.dropna(axis=1)
```

Out[93]:

	Name	AT	S.St	Hindi
0	Raman	1	18	20
1	Raman	2	17	22
2	Raman	3	15	24
3	Raman	4	19	18
4	Charan	1	22	24
5	Charan	2	21	25
6	Charan	3	19	23
7	Charan	4	17	21
8	Ajay	1	20	15
9	Ajay	2	24	17
10	Ajay	3	19	21
11	Ajay	4	20	20
12	Madhu	1	25	22
13	Madhu	2	25	24
14	Madhu	3	20	25
15	Madhu	4	19	20

In [94]:
```
df1.dropna(axis=0)
```

Out[94]:

	Name	AT	Maths	Science	S.St	Hindi	Eng
0	Raman	1	22.0	21.0	18	20	21.0
1	Raman	2	21.0	20.0	17	22	24.0
2	Raman	3	14.0	19.0	15	24	23.0
4	Charan	1	20.0	17.0	22	24	19.0
5	Charan	2	23.0	15.0	21	25	15.0
6	Charan	3	22.0	18.0	19	23	13.0
7	Charan	4	19.0	20.0	17	21	16.0
8	Ajay	1	23.0	19.0	20	15	22.0
9	Ajay	2	24.0	22.0	24	17	21.0
10	Ajay	3	12.0	25.0	19	21	23.0
11	Ajay	4	15.0	20.0	20	20	17.0
12	Madhu	1	15.0	22.0	25	22	22.0
13	Madhu	2	18.0	21.0	25	24	23.0
14	Madhu	3	17.0	18.0	20	25	20.0
15	Madhu	4	14.0	20.0	19	20	18.0

In [95]:
```
df1["Science"].isnull()
```

Out[95]:
```
0     False
1     False
2     False
3      True
4     False
5     False
6     False
7     False
8     False
9     False
10    False
11    False
12    False
13    False
14    False
15    False
Name: Science, dtype: bool
```

In [96]:
```
'''To check whether a column (attribute) has a missing
value in the entire dataset, any() function is used. It
returns True in case of missing value else returns False'''
df1.isnull().any()
```

Out[96]:
```
Name       False
AT         False
Maths       True
Science     True
S.St       False
Hindi      False
Eng         True
dtype: bool
```

In [101]:
```
a=df1[df1["Name"]=="Raman"]
```

In [102]:
```
a
```

Out[102]:

	Name	AT	Maths	Science	S.St	Hindi	Eng
0	Raman	1	22.0	21.0	18	20	21.0
1	Raman	2	21.0	20.0	17	22	24.0
2	Raman	3	14.0	19.0	15	24	23.0
3	Raman	4	NaN	NaN	19	18	NaN

In [103]:
```
a["Hindi"].sum()
```

Out[103]:
```
84
```

In [104]:
```
#find the percentage of marks scored by raman in hindi
```

In [105]:
```
#Dropping missing values
```

In [106]:
```
'''Dropping will remove the entire row (object) having
the missing value(s). This strategy reduces the size of
the dataset used in data analysis, hence should be used
in case of missing values on few objects.'''
```

Out[106]:
```
'Dropping will remove the entire row (object) having \nthe missing value(s). This strategy reduce
s the size of \nthe dataset used in data analysis, hence should be used \nin case of missing valu
es on few objects.'
```

In [107]:
```
df1.dropna()
```

Out[107]:

	Name	AT	Maths	Science	S.St	Hindi	Eng
0	Raman	1	22.0	21.0	18	20	21.0
1	Raman	2	21.0	20.0	17	22	24.0
2	Raman	3	14.0	19.0	15	24	23.0
4	Charan	1	20.0	17.0	22	24	19.0
5	Charan	2	23.0	15.0	21	25	15.0
6	Charan	3	22.0	18.0	19	23	13.0
7	Charan	4	19.0	20.0	17	21	16.0
8	Ajay	1	23.0	19.0	20	15	22.0
9	Ajay	2	24.0	22.0	24	17	21.0
10	Ajay	3	12.0	25.0	19	21	23.0
11	Ajay	4	15.0	20.0	20	20	17.0
12	Madhu	1	15.0	22.0	25	22	22.0
13	Madhu	2	18.0	21.0	25	24	23.0
14	Madhu	3	17.0	18.0	20	25	20.0
15	Madhu	4	14.0	20.0	19	20	18.0

In [108]:
```
#Estimating Missing Values
'''Missing values can be filled by using estimations or
approximations The fillna(num) function can be used to replace
missing value(s) by the value specified in num. For
example, fillna(0) replaces missing value by 0. Similarly
fillna(1) replaces missing value by 1.'''
```

Out[108]:
```
'Missing values can be filled by using estimations or \napproximations The fillna(num) function c
an be used to replace \nmissing value(s) by the value specified in num. For \nexample, fillna(0)
replaces missing value by 0. Similarly \nfillna(1) replaces missing value by 1.'
```

In [109]:
```
df1.fillna(0)
```

Out[109]:

	Name	AT	Maths	Science	S.St	Hindi	Eng
0	Raman	1	22.0	21.0	18	20	21.0
1	Raman	2	21.0	20.0	17	22	24.0
2	Raman	3	14.0	19.0	15	24	23.0
3	Raman	4	0.0	0.0	19	18	0.0
4	Charan	1	20.0	17.0	22	24	19.0
5	Charan	2	23.0	15.0	21	25	15.0
6	Charan	3	22.0	18.0	19	23	13.0
7	Charan	4	19.0	20.0	17	21	16.0
8	Ajay	1	23.0	19.0	20	15	22.0
9	Ajay	2	24.0	22.0	24	17	21.0
10	Ajay	3	12.0	25.0	19	21	23.0
11	Ajay	4	15.0	20.0	20	20	17.0
12	Madhu	1	15.0	22.0	25	22	22.0
13	Madhu	2	18.0	21.0	25	24	23.0
14	Madhu	3	17.0	18.0	20	25	20.0
15	Madhu	4	14.0	20.0	19	20	18.0

In [110]:
```
#forward fill takes missing values from previous row
df1.ffill()
```

Out[110]:

	Name	AT	Maths	Science	S.St	Hindi	Eng
0	Raman	1	22.0	21.0	18	20	21.0
1	Raman	2	21.0	20.0	17	22	24.0
2	Raman	3	14.0	19.0	15	24	23.0
3	Raman	4	14.0	19.0	19	18	23.0
4	Charan	1	20.0	17.0	22	24	19.0
5	Charan	2	23.0	15.0	21	25	15.0
6	Charan	3	22.0	18.0	19	23	13.0
7	Charan	4	19.0	20.0	17	21	16.0
8	Ajay	1	23.0	19.0	20	15	22.0
9	Ajay	2	24.0	22.0	24	17	21.0
10	Ajay	3	12.0	25.0	19	21	23.0
11	Ajay	4	15.0	20.0	20	20	17.0
12	Madhu	1	15.0	22.0	25	22	22.0
13	Madhu	2	18.0	21.0	25	24	23.0
14	Madhu	3	17.0	18.0	20	25	20.0
15	Madhu	4	14.0	20.0	19	20	18.0

In [111]:
```
#imputing (assign) missing values with mean
#We calculate the mean of the 'Age' column using mean() and then use fillna() to replace missing va
a=df1["Maths"].mean()
```

In [112]:
```
df1.fillna({"Maths":a})
```

Out[112]:

	Name	AT	Maths	Science	S.St	Hindi	Eng
0	Raman	1	22.0	21.0	18	20	21.0
1	Raman	2	21.0	20.0	17	22	24.0
2	Raman	3	14.0	19.0	15	24	23.0
3	Raman	4	18.6	NaN	19	18	NaN
4	Charan	1	20.0	17.0	22	24	19.0
5	Charan	2	23.0	15.0	21	25	15.0
6	Charan	3	22.0	18.0	19	23	13.0
7	Charan	4	19.0	20.0	17	21	16.0
8	Ajay	1	23.0	19.0	20	15	22.0
9	Ajay	2	24.0	22.0	24	17	21.0
10	Ajay	3	12.0	25.0	19	21	23.0
11	Ajay	4	15.0	20.0	20	20	17.0
12	Madhu	1	15.0	22.0	25	22	22.0
13	Madhu	2	18.0	21.0	25	24	23.0
14	Madhu	3	17.0	18.0	20	25	20.0
15	Madhu	4	14.0	20.0	19	20	18.0

In [113]:
```
#binning
''''dividing the data into bins
results = [12,34,67,55,28,90,99,12,3,56,74,44,87,23,49,89,87]
You know that the experimental values have a range from 0 to 100; therefore you can uniformly divi
this interval, for example, into four equal parts, i.e., bins. The first contains the values betwe
second between 26 and 50, the third between 51 and 75, and the last between 76 and 100.'''
```

Out[113]:
```
"'dividing the data into bins\nresults = [12,34,67,55,28,90,99,12,3,56,74,44,87,23,49,89,87]\nYou
know that the experimental values have a range from 0 to 100; therefore you can uniformly divide
\nthis interval, for example, into four equal parts, i.e., bins. The first contains the values be
tween 0 and 25, the \nsecond between 26 and 50, the third between 51 and 75, and the last between
76 and 100."
```

In [114]:
```
import pandas as pd
results=[0,12,34,67,55,28,90,99,12,3,56,74,44,87,23,49,89,87,105]
```

In [115]:
```
 bins = [0,25,50,75,100]
```

In [116]:
```
b= pd.cut(results, bins)
b
```

Out[116]:
```
[NaN, (0.0, 25.0], (25.0, 50.0], (50.0, 75.0], (50.0, 75.0], ..., (0.0, 25.0], (25.0, 50.0], (75.
0, 100.0], (75.0, 100.0], NaN]
Length: 19
Categories (4, interval[int64, right]): [(0, 25] < (25, 50] < (50, 75] < (75, 100]]
```

In [117]:
```
bin_names=["fail","pass","disction","excellent"]
```

In [118]:
```
 pd.cut(results, bins, labels=bin_names)
```

Out[118]:
```
[NaN, 'fail', 'pass', 'disction', 'disction', ..., 'fail', 'pass', 'excellent', 'excellent', NaN]
Length: 19
Categories (4, object): ['fail' < 'pass' < 'disction' < 'excellent']
```

In [119]:
```
#qcut(). This function divides the
#sample directly into quintiles.
b= pd.qcut(results, 4)
```

In [120]:
```
b
```

Out[120]:
```
[(-0.001, 25.5], (-0.001, 25.5], (25.5, 55.0], (55.0, 87.0], (25.5, 55.0], ..., (-0.001, 25.5],
(25.5, 55.0], (87.0, 105.0], (55.0, 87.0], (87.0, 105.0]]
Length: 19
Categories (4, interval[float64, right]): [(-0.001, 25.5] < (25.5, 55.0] < (55.0, 87.0] < (87.0,
105.0]]
```

In [ ]:
```

```

In [121]:
```
#CORRELATION
```

In [123]:
```
'''Correlation analysis is used to measure the strength and direction of a
linear relationship between two numeric variables.
In Python, the pandas library provides a convenient way to calculate
correlation coefficients using the corr method.'''
```

Out[123]:
```
'Correlation analysis is used to measure the strength and direction of a \nlinear relationship be
tween two numeric variables. \nIn Python, the pandas library provides a convenient way to calcula
te \ncorrelation coefficients using the corr method.'
```

In [124]:
```
import pandas as pd

# Create DataFrame
data = {'Feature1': [1, 2, 3, 4, 5],
        'Feature2': [5, 4, 3, 2, 1],
        'Feature3': [2, 3, 2, 4, 1]}
df = pd.DataFrame(data)

# Calculate correlation matrix
correlation_matrix = df.corr()

# Display correlation matrix
print("Correlation Matrix:")
print(correlation_matrix)
```

```
Correlation Matrix:
          Feature1  Feature2  Feature3
Feature1  1.000000 -1.000000 -0.138675
Feature2 -1.000000  1.000000  0.138675
Feature3 -0.138675  0.138675  1.000000
```

In [126]:
```
'''The values in the matrix range from -1 to 1, where:

1 indicates a perfect positive correlation,
-1 indicates a perfect negative correlation, and
0 indicates no correlation.'''
```

Out[126]:
```
'The values in the matrix range from -1 to 1, where:\n\n1 indicates a perfect positive correlatio
n,\n-1 indicates a perfect negative correlation, and\n0 indicates no correlation.'
```

In [128]:
```
'''A perfect positive correlation means that two variables move in
the same direction with a constant proportionality.
In other words, as one variable increases, the other variable
also increases, and the relationship between them is linear and exact.
The correlation coefficient in this case would be +1.'''
```

Out[128]:
```
'A perfect positive correlation means that two variables move in \nthe same direction with a cons
tant proportionality.\nIn other words, as one variable increases, the other variable \nalso incre
ases, and the relationship between them is linear and exact. \nThe correlation coefficient in thi
s case would be +1.'
```

In [131]:
```
'''A perfect negative correlation means that two variables
move in opposite directions with a constant proportionality.
In other words, as one variable increases, the other variable decreases,
and the relationship between them is linear and exact.
The correlation coefficient in this case would be -1.'''
```

Out[131]:
```
'A perfect negative correlation means that two variables\nmove in opposite directions with a cons
tant proportionality.\nIn other words, as one variable increases, the other variable decreases,
\nand the relationship between them is linear and exact. \nThe correlation coefficient in this ca
se would be -1.'
```

In [132]:
```
#assignment find the correlation between rguktrkv and rguktnuzvid incase of placements from 2016 t
```

In [133]:
```
#assignment on dataframe , importing and exporting
#create your own dataset(data frame) that contains your 10th marks,puc1,pu2,e1,e2 marks in all sub
# then save your marks in an excel file named "your id number".
```

In [134]:
```
#assignment on dataframe create education backgound table with col names degree/course,institute,ye
```

In [135]:
```
#grouping
#group by() function
```

In [136]:
```
'''In pandas, DataFrame.GROUP BY() function is used
to split the data into groups based on some criteria.
Pandas objects like a DataFrame can be split on any
of their axes. The GROUP BY function works based on
a split-apply-combine strategy which is shown below
using a 3-step process:
Step 1: Split the data into groups by creating a GROUP
BY object from the original DataFrame.
Step 2: Apply the required function.
Step 3: Combine the results to form a new DataFrame
group is generally performed to do some aggregate operations like sum(),
min(),max(),mean(),var() means variance, std() means standard deviation etc'''
```

Out[136]:
```
'In pandas, DataFrame.GROUP BY() function is used \nto split the data into groups based on some c
riteria. \nPandas objects like a DataFrame can be split on any \nof their axes. The GROUP BY func
tion works based on \na split-apply-combine strategy which is shown below \nusing a 3-step proces
s:\nStep 1: Split the data into groups by creating a GROUP \nBY object from the original DataFram
e.\nStep 2: Apply the required function. \nStep 3: Combine the results to form a new DataFrame\ng
roup is generally performed to do some aggregate operations like sum(),\nmin(),max(),mean(),var()
means variance, std() means standard deviation etc'
```

In [137]:
```
df1
```

Out[137]:

	Name	AT	Maths	Science	S.St	Hindi	Eng
0	Raman	1	22.0	21.0	18	20	21.0
1	Raman	2	21.0	20.0	17	22	24.0
2	Raman	3	14.0	19.0	15	24	23.0
3	Raman	4	NaN	NaN	19	18	NaN
4	Charan	1	20.0	17.0	22	24	19.0
5	Charan	2	23.0	15.0	21	25	15.0
6	Charan	3	22.0	18.0	19	23	13.0
7	Charan	4	19.0	20.0	17	21	16.0
8	Ajay	1	23.0	19.0	20	15	22.0
9	Ajay	2	24.0	22.0	24	17	21.0
10	Ajay	3	12.0	25.0	19	21	23.0
11	Ajay	4	15.0	20.0	20	20	17.0
12	Madhu	1	15.0	22.0	25	22	22.0
13	Madhu	2	18.0	21.0	25	24	23.0
14	Madhu	3	17.0	18.0	20	25	20.0
15	Madhu	4	14.0	20.0	19	20	18.0

In [138]:
```
g1=df1.groupby('Name')
```

In [139]:
```
g1
```

Out[139]:
```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x000001F6AD15A5D0>
```

In [140]:
```
g1.first()
```

Out[140]:

Name	AT	Maths	Science	S.St	Hindi	Eng
Ajay	1	23.0	19.0	20	15	22.0
Charan	1	20.0	17.0	22	24	19.0
Madhu	1	15.0	22.0	25	22	22.0
Raman	1	22.0	21.0	18	20	21.0

In [141]:
```
#Displaying the first entry from each group

g1.size()#to display the size of each group
```

Out[141]:
```
Name
Ajay      4
Charan    4
Madhu     4
Raman     4
dtype: int64
```

In [142]:
```
#Printing data of a single group
g1.get_group("Raman")
```

Out[142]:

	Name	AT	Maths	Science	S.St	Hindi	Eng
0	Raman	1	22.0	21.0	18	20	21.0
1	Raman	2	21.0	20.0	17	22	24.0
2	Raman	3	14.0	19.0	15	24	23.0
3	Raman	4	NaN	NaN	19	18	NaN

In [143]:
```
#Grouping with respect to multiple attributes
#Creating a GROUP BY Name and UT
g2=df1.groupby(["Name","AT"])
g2
```

Out[143]:
```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x000001F6AD203A90>
```

In [144]:
```
g2.first()
```

Out[144]:

		Maths	Science	S.St	Hindi	Eng
Name	**AT**					
Ajay	**1**	23.0	19.0	20	15	22.0
	2	24.0	22.0	24	17	21.0
	3	12.0	25.0	19	21	23.0
	4	15.0	20.0	20	20	17.0
Charan	**1**	20.0	17.0	22	24	19.0
	2	23.0	15.0	21	25	15.0
	3	22.0	18.0	19	23	13.0
	4	19.0	20.0	17	21	16.0
Madhu	**1**	15.0	22.0	25	22	22.0
	2	18.0	21.0	25	24	23.0
	3	17.0	18.0	20	25	20.0
	4	14.0	20.0	19	20	18.0
Raman	**1**	22.0	21.0	18	20	21.0
	2	21.0	20.0	17	22	24.0
	3	14.0	19.0	15	24	23.0
	4	NaN	NaN	19	18	NaN

In [145]:
```
g1.sum()
```

Out[145]:

	AT	Maths	Science	S.St	Hindi	Eng
Name						
Ajay	10	74.0	86.0	83	73	83.0
Charan	10	84.0	70.0	79	93	63.0
Madhu	10	64.0	81.0	89	91	83.0
Raman	10	57.0	60.0	69	84	68.0

In [146]:
```
g1.count()
```

Out[146]:

	AT	Maths	Science	S.St	Hindi	Eng
Name						
Ajay	4	4	4	4	4	4
Charan	4	4	4	4	4	4
Madhu	4	4	4	4	4	4
Raman	4	3	3	4	4	3

In [147]:
```
g1.max()
```

Out[147]:

Name	AT	Maths	Science	S.St	Hindi	Eng
Ajay	4	24.0	25.0	24	21	23.0
Charan	4	23.0	20.0	22	25	19.0
Madhu	4	18.0	22.0	25	25	23.0
Raman	4	22.0	21.0	19	24	24.0

In [148]:
```
# Aggregation can be performed
#using agg() or aggregate() function also
```

In [149]:
```
g1.agg("mean")
```

Out[149]:

Name	AT	Maths	Science	S.St	Hindi	Eng
Ajay	2.5	18.5	21.50	20.75	18.25	20.750000
Charan	2.5	21.0	17.50	19.75	23.25	15.750000
Madhu	2.5	16.0	20.25	22.25	22.75	20.750000
Raman	2.5	19.0	20.00	17.25	21.00	22.666667

In [150]:
```
g1.aggregate(["mean","var","std"])
```

Out[150]:

	AT			Maths			Science			S.St			Hi
	mean	var	std	mean	var	std	mean	var	std	mean	var	std	me
Name													
Ajay	2.5	1.666667	1.290994	18.5	35.000000	5.916080	21.50	7.000000	2.645751	20.75	4.916667	2.217356	18
Charan	2.5	1.666667	1.290994	21.0	3.333333	1.825742	17.50	4.333333	2.081666	19.75	4.916667	2.217356	23
Madhu	2.5	1.666667	1.290994	16.0	3.333333	1.825742	20.25	2.916667	1.707825	22.25	10.250000	3.201562	22
Raman	2.5	1.666667	1.290994	19.0	19.000000	4.358899	20.00	1.000000	1.000000	17.25	2.916667	1.707825	21

In [ ]:
```

```

In [151]:
```
#create a dataframe that contains atleast 5 student names and their assessment test marks in all f
```

In [154]:
```
'''Descriptive Statistics
Descriptive Statistics are used to summarise the given
data. In other words, they refer to the methods which
are used to get some basic idea about the data.
we will be discussing descriptive
statistical methods that can be applied to a DataFrame.
These are max, min, count, sum, mean, median, mode,
quartiles, variance'''
```

Out[154]:
```
'Descriptive Statistics\nDescriptive Statistics are used to summarise the given \ndata. In other
words, they refer to the methods which \nare used to get some basic idea about the data. \nwe wil
l be discussing descriptive \nstatistical methods that can be applied to a DataFrame. \nThese are
max, min, count, sum, mean, median, mode, \nquartiles, variance'
```

In [155]:
```
df1
```

Out[155]:

	Name	AT	Maths	Science	S.St	Hindi	Eng
0	Raman	1	22.0	21.0	18	20	21.0
1	Raman	2	21.0	20.0	17	22	24.0
2	Raman	3	14.0	19.0	15	24	23.0
3	Raman	4	NaN	NaN	19	18	NaN
4	Charan	1	20.0	17.0	22	24	19.0
5	Charan	2	23.0	15.0	21	25	15.0
6	Charan	3	22.0	18.0	19	23	13.0
7	Charan	4	19.0	20.0	17	21	16.0
8	Ajay	1	23.0	19.0	20	15	22.0
9	Ajay	2	24.0	22.0	24	17	21.0
10	Ajay	3	12.0	25.0	19	21	23.0
11	Ajay	4	15.0	20.0	20	20	17.0
12	Madhu	1	15.0	22.0	25	22	22.0
13	Madhu	2	18.0	21.0	25	24	23.0
14	Madhu	3	17.0	18.0	20	25	20.0
15	Madhu	4	14.0	20.0	19	20	18.0

In [156]:
```
'''the maximum value of
each column of the DataFrame: by default the methods min,max,sum,mode,count,std,var returns the max
df1.max()
```

Out[156]:
```
Name       Raman
AT             4
Maths       24.0
Science     25.0
S.St          25
Hindi         25
Eng         24.0
dtype: object
```

In [ ]:
```

```

In [157]:
```

df1.max(axis=0)
```

Out[157]:
```
Name       Raman
AT             4
Maths       24.0
Science     25.0
S.St          25
Hindi         25
Eng         24.0
dtype: object
```

In [158]:
```
df2=pd.DataFrame([{1:10,2:20,3:30},{1:11,2:12,3:13}])

df2.min(axis=0)
```

Out[158]:
```
1    10
2    12
3    13
dtype: int64
```

In [159]:
```
df1.sum()
```

Out[159]:
```
Name       RamanRamanRamanRamanCharanCharanCharanCharanAj...
AT                                                        40
Maths                                                  279.0
Science                                                297.0
S.St                                                     320
Hindi                                                    341
Eng                                                    297.0
dtype: object
```

In [160]:
```
df2.sum(axis=1)
```

Out[160]:
```
0    60
1    36
dtype: int64
```

In [161]:
```
df1["Science"].sum()
```

Out[161]:
```
297.0
```

In [162]:
```
#calculating no.of values
df1.count()
```

Out[162]:
```
Name       16
AT         16
Maths      15
Science    15
S.St       16
Hindi      16
Eng        15
dtype: int64
```

In [163]:
```
#count the number of values in a row
df1.count(axis=1)
```

Out[163]:
```
0     7
1     7
2     7
3     4
4     7
5     7
6     7
7     7
8     7
9     7
10    7
11    7
12    7
13    7
14    7
15    7
dtype: int64
```

In [164]:
```
'''Calculating Mean
DataFrame.mean() will display the mean (average) of
the values of each column of a DataFrame. It is only
applicable for numeric values. '''
df2.mean()
```

Out[164]:
```
1    10.5
2    16.0
3    21.5
dtype: float64
```

In [165]:
```
df2.mean(axis=1)
```

Out[165]:
```
0    20.0
1    12.0
dtype: float64
```

In [166]:
```
'''Calculating Median
DataFrame.Median() will display the middle value of the
data. This function will display the median of the values
of each column of a DataFrame. It is only applicable for
numeric values.'''
print(df2.median())
```

```
1    10.5
2    16.0
3    21.5
dtype: float64
```

In [167]:
```
'''DateFrame.mode() will display the mode. The mode is
defined as the value that appears the most number of
times in a data. This function will display the mode of
each column or row of the DataFrame. To get the mode
'''
df2.mode()
```

Out[167]:

	1	2	3
0	10	12	13
1	11	20	30

In [168]:
```
'''Calculating Variance
DataFrame.var() is used to display the variance. It is the
average of squared differences from the mean.'''
df2.var()
```

Out[168]:
```
1      0.5
2     32.0
3    144.5
dtype: float64
```

In [169]:
```
df1[['Maths','Science','S.St','Hindi','Eng']].var()
```

Out[169]:
```
Maths      15.257143
Science     5.600000
S.St        8.133333
Hindi       8.495833
Eng        11.171429
dtype: float64
```

In [170]:
```
df3=pd.DataFrame([{"sub1":98,"sub2":95,"sub3":85},{"sub1":93,"sub2":91,"sub3":89},{"sub1":82,"sub2
df3
```

Out[170]:

	sub1	sub2	sub3
0	98	95	85
1	93	91	89
2	82	97	95

In [171]:
```
df3.loc[0].sum()
df3.loc[0].var()
df3.min(axis=1)
```

Out[171]:
```
0    85
1    89
2    82
dtype: int64
```

In [172]:
```
'''DataFrame.std() returns the standard deviation of the
values. Standard deviation is calculated as the square
root of the variance.'''
df1[['Maths','Science','S.St','Hindi','Eng']].std()
```

Out[172]:
```
Maths      3.906039
Science    2.366432
S.St       2.851900
Hindi      2.914761
Eng        3.342369
dtype: float64
```

df1.describe()

In [173]:
```
df1.describe()
```

Out[173]:

	AT	Maths	Science	S.St	Hindi	Eng
count	16.000000	15.000000	15.000000	16.0000	16.000000	15.000000
mean	2.500000	18.600000	19.800000	20.0000	21.312500	19.800000
std	1.154701	3.906039	2.366432	2.8519	2.914761	3.342369
min	1.000000	12.000000	15.000000	15.0000	15.000000	13.000000
25%	1.750000	15.000000	18.500000	18.7500	20.000000	17.500000
50%	2.500000	19.000000	20.000000	19.5000	21.500000	21.000000
75%	3.250000	22.000000	21.000000	21.2500	24.000000	22.500000
max	4.000000	24.000000	25.000000	25.0000	25.000000	24.000000

In [174]:
```
'''DataFrame.describe() function displays the
descriptive statistical values in a single command. These
values help us describe a set of data in a DataFrame.'''
df1.describe()
```

Out[174]:

	AT	Maths	Science	S.St	Hindi	Eng
count	16.000000	15.000000	15.000000	16.0000	16.000000	15.000000
mean	2.500000	18.600000	19.800000	20.0000	21.312500	19.800000
std	1.154701	3.906039	2.366432	2.8519	2.914761	3.342369
min	1.000000	12.000000	15.000000	15.0000	15.000000	13.000000
25%	1.750000	15.000000	18.500000	18.7500	20.000000	17.500000
50%	2.500000	19.000000	20.000000	19.5000	21.500000	21.000000
75%	3.250000	22.000000	21.000000	21.2500	24.000000	22.500000
max	4.000000	24.000000	25.000000	25.0000	25.000000	24.000000

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [1]:
```
"""Data formatting refers to the process of organizing and structuring raw data
into a specific layout that is suitable for analysis, visualization, or
further processing. Properly formatted data is crucial for accurate and
efficient analysis.
In Python, there are several libraries and tools available for
data formatting. One of the most commonly used libraries
for data manipulation and formatting is pandas."""
```

Out[1]:
```
'Data formatting refers to the process of organizing and structuring raw data
\ninto a specific layout that is suitable for analysis, visualization, or \nf
urther processing. Properly formatted data is crucial for accurate and \neffi
cient analysis. \nIn Python, there are several libraries and tools available
for \ndata formatting. One of the most commonly used libraries \nfor data man
ipulation and formatting is pandas.'
```

In [2]:
```
#Basic Data Formatting with Pandas:
#1.Loading Data:
    df = pd.read_csv('your_data.csv')
```

```
  Cell In[2], line 3
    df = pd.read_csv('your_data.csv')
    ^
IndentationError: unexpected indent
```

In [ ]:
```
#2.Handling Missing Values:Remove or impute missing values.
# Drop rows with missing values
df = df.dropna()

# Impute missing values with mean
df = df.fillna(df.mean())
```

In [ ]:
```
```

In [3]:
```
# 3. Data Types: Check and set the data types of columns.
# Check data types
import pandas as pd
df=pd.DataFrame([{"sub1":26,"sub2":31,"sub3":35},{"sub1":11,"sub2":19,"sub3":25
print(df.dtypes)
```

```
sub1    int64
sub2    int64
sub3    int64
dtype: object
```

In [4]:
```
# Convert a column to a specific data type
df['sub1'] = df['sub1'].astype('float')
```

In [5]:
```
df.dtypes
```

Out[5]:
```
sub1    float64
sub2      int64
sub3      int64
dtype: object
```

In [6]:
```
#4. Renaming Columns:Rename columns for clarity.
df.rename({"sub1":"subject1"},axis="columns")
```

Out[6]:

	subject1	sub2	sub3
0	26.0	31	35
1	11.0	19	25
2	15.0	26	23

In [7]:
```
#Sorting and Reordering:

#Sort or reorder rows and columns.
df.sort_values(by='sub1')
```

Out[7]:

	sub1	sub2	sub3
1	11.0	19	25
2	15.0	26	23
0	26.0	31	35

In [8]:
```
df.rename({0:"at1",1:"at2",2:"at3"},axis="index")
```

Out[8]:

	sub1	sub2	sub3
at1	26.0	31	35
at2	11.0	19	25
at3	15.0	26	23

In [9]:
```
# Reorder columns
df[['sub2', 'sub3', 'sub1']]
```

Out[9]:

	sub2	sub3	sub1
0	31	35	26.0
1	19	25	11.0
2	26	23	15.0

In [10]:
```
#reorder rows
df.iloc[[1,0,2]]
```

Out[10]:

	sub1	sub2	sub3
1	11.0	19	25
0	26.0	31	35
2	15.0	26	23

In [11]:
```
5.Exporting Data:

Save the formatted DataFrame to a new file.
python
Copy code
# Save to CSV
df.to_csv('formatted_data.csv')
```

```
  Cell In[11], line 1
    5.Exporting Data:
     ^
SyntaxError: invalid decimal literal
```

In [ ]:
```

```

In [1]:
```
'''ANOVA is a statistical method used to analyze the differences
among group means in a sample. It assesses whether there are any
statistically significant differences between the means of three or
more independent groups. ANOVA compares the variance within each group
to the variance between groups, and if the between-group variance is
significantly larger than the within-group variance,
it suggests that there are significant differences between the groups.'''
```

Out[1]:
```
'ANOVA is a statistical method used to analyze the differences \namong group
means in a sample. It assesses whether there are any \nstatistically signific
ant differences between the means of three or \nmore independent groups. ANOV
A compares the variance within each group \nto the variance between groups, a
nd if the between-group variance is \nsignificantly larger than the within-gr
oup variance, \nit suggests that there are significant differences between th
e groups.'
```

In [*]:
```
import scipy.stats as stats
```

In [*]:
```
group1 = [68, 72, 65, 74, 71]
group2 = [60, 65, 68, 63, 70]
group3 = [75, 78, 82, 79, 85]
```

In [*]:
```
f_statistic, p_value = stats.f_oneway(group1, group2,group3)
```

In [*]:
```
print("F-Statistic:", f_statistic)
```

In [*]:
```
print("P-Value:", p_value)
```

In [*]:
```
if p_value < 0.05:
    print("Reject the null hypothesis. There is a significant difference betwee
else:
    print("Fail to reject the null hypothesis. There is no significant differen
```

In [*]:
```
'''the stats.f_oneway() function in the scipy.stats module is used to perform
a one-way analysis of variance (ANOVA). It compares the means of two or more
independent groups to determine whether there are statistically significant
differences among them.
The function returns an F-statistic and a p-value.'''
```

In [ ]:
```
'''f_statistic: This is the F-statistic, which is a ratio of
the variances between groups to the variances within groups.
A larger F-statistic suggests a greater difference between group
means relative to the variability within groups.

p_value: This is the p-value associated with the F-statistic.
It represents the probability of observing the data if the null hypothesis
(i.e., all group means are equal) is true.
A smaller p-value (typically less than 0.05)
suggests evidence against the null hypothesis.'''
```

In [ ]:
```

```